

■ JAVA METHODS MASTER PROJECT

Ultimate Student Survival System — 3-Hour Classroom Project

A funny but serious Java project combining concepts learned up to Methods: variables, Scanner, operators, conditions, loops, break, counters/accumulators, methods, parameters, arguments, return, void, scope, static, method composition and overloading.

■ Project Goal

Build one application that creates a student's **College + Love Life + Money Survival Report**. Students first design the logic, then implement each module as reusable methods.

Module	Tasks	Methods
■ Academic	Attendance, average, eligibility	calculateAttendance(), calculateAverage(), checkEligibility()
♥■ Love Life	Compatibility score + status	calculateCompatibility(), relationshipStatus()
■ Money	Expenses, savings, budget	calculateExpenses(), calculateSavings(), checkBudget()
■ Final Report	Combined score + final message	calculateLifeScore(), getFinalStatus(), printReport()

■ Concepts Checklist

1. Variables & Data Types
2. Scanner input
3. Arithmetic / Relational / Logical / Assignment operators
4. if / if-else / else-if / nested if
5. for / while
6. break
7. Counters / Accumulators
8. void methods
9. return methods
10. Parameters & Arguments
11. Local scope
12. static
13. Method Composition
14. Method Overloading

■■ 3-Hour Classroom Plan

0–15	Story + requirements	Inputs, outputs, modules
15–35	Scanner + variables	Data types and validation discussion
35–60	Academic module	conditions + calculations + return
60–80	Love module	parameters + return + String result
80–90	Break / dry run	Trace values
90–115	Money module	accumulator + methods
115–135	Loop challenge	while + sentinel 0 + break
135–150	Composition	methods calling methods
150–165	Overloading	same name, different parameters
165–175	Integration	clean main()
175–180	Demo + exit questions	Explain one method

■ PHASE 1 — Academic Module

Inputs: attended classes, total classes, three subject marks.

Calculate attendance %, average marks, and eligibility. Eligible when attendance $\geq 75\%$ AND average ≥ 40 .

Suggested methods:

```
static double calculateAttendance(int attended, int total)
static double calculateAverage(int m1, int m2, int m3)
static boolean checkEligibility(double attendance, double average)
static void printAcademicResult(double attendance, double average, boolean eligible)
```

Ask: **Why return a value instead of printing from calculateAttendance()?**

♥■ PHASE 2 — Love Life Compatibility

Inputs: patience (1–10), reply time in minutes, monthly relationship budget, number of arguments.

Start score at 50. Add/subtract points using conditions. Then convert the score to a funny status.

```
static int calculateCompatibility(int patience, int replyTime, int budget, int arguments)
static String relationshipStatus(int score)
static void printRelationshipResult(int score, String status)
```

Example: **Compatibility Score: 78 — Relationship Material ♥■ — Warning: Arguments thode kam karo ■**

Loop extension: test up to 3 cases; use a counter and allow break to stop early.

■ PHASE 3 — Money Survival

Inputs: income, rent, food, travel, recharge, entertainment. Calculate total expenses and savings.

```
static int calculateExpenses(int rent, int food, int travel, int recharge, int entertainment)
static int calculateSavings(int income, int expenses)
static boolean checkBudget(int savings)
static void printBudgetResult(int income, int expenses, int savings)
```

Funny rules: savings < 0 → “Bhai, salary se pehle expenses aa gaye ■”; savings == 0 → “Balance: Zen mode ■”; savings > 5000 → “Future CEO detected ■”.

■ PHASE 4 — Loop + Accumulator Challenge

Let the student enter food-item prices repeatedly. Enter **0** to stop. Do not add the 0 to the total.

```
int totalFood = 0;
while (true) {
    int price = sc.nextInt();
    if (price == 0) break;
    totalFood = totalFood + price;
}
```

Pass totalFood into the expense method. This demonstrates **loop** → **collect data** → **method** → **return** → **reuse**.

■ PHASE 5 — Method Composition

The final score method should combine the results instead of repeating calculations.

```
static int calculateLifeScore(double attendance, double average,  
int compatibility, int savings) {  
    int score = 0;  
    if (attendance >= 75) score += 25;  
    if (average >= 60) score += 25;  
    if (compatibility >= 70) score += 25;  
    if (savings >= 5000) score += 25;  
    return score;  
}
```

Key idea: **One method = one clear responsibility.**

■ PHASE 6 — Method Overloading

```
static void printReport() { ... }  
static void printReport(int score) { ... }  
static void printReport(int score, String status) { ... }
```

Ask: Why is this valid? Why can't changing only the return type create a valid overload?

■■ FINAL CODE SKELETON

```
import java.util.Scanner;

public class StudentSurvivalSystem {

    static double calculateAttendance(int attended, int total) {
        return (attended * 100.0) / total;
    }

    static double calculateAverage(int a, int b, int c) {
        return (a + b + c) / 3.0;
    }

    static boolean checkEligibility(double attendance, double average) {
        return attendance >= 75 && average >= 40;
    }

    static int calculateCompatibility(int patience, int replyTime, int budget, int arguments) {
        int score = 50;
        if (patience >= 7) score += 15;
        if (replyTime <= 30) score += 15;
        if (budget >= 2000) score += 10;
        if (arguments <= 3) score += 10;
        return score;
    }

    static int calculateExpenses(int rent, int food, int travel, int recharge, int entertainment) {
        return rent + food + travel + recharge + entertainment;
    }

    static int calculateSavings(int income, int expenses) { return income - expenses; }

    static int calculateLifeScore(double attendance, double average, int compatibility, int savings) {
        int score = 0;
        if (attendance >= 75) score += 25;
        if (average >= 60) score += 25;
        if (compatibility >= 70) score += 25;
        if (savings >= 5000) score += 25;
        return score;
    }

    static String getFinalStatus(int score) {
        if (score >= 75) return "Survival Master ■";
        else if (score >= 50) return "Needs Improvement ■";
        else return "Save Yourself First ■";
    }

    static void printReport(int score, String status) {
```

```
System.out.println("==== FINAL SURVIVAL REPORT =====");
System.out.println("Life Score: " + score);
System.out.println("Status: " + status);
}

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
// Students integrate all inputs and method calls here.
}
}
```

■ COMMON LOOPHOLES / DEBUG CHECKLIST

1. Use 100.0 for attendance percentage to avoid integer division.
2. `total == 0` must not be allowed before dividing.
3. Every non-void method needs a return value on every path.
4. Parameters are local to their method; don't use them outside it.
5. Return a calculation when the caller needs it; don't only print it.
6. Changing only return type does not overload a method.
7. Sentinel 0 ends the loop and should not be added to an accumulator.
8. Update the loop variable or the loop can become infinite.
9. Do not duplicate the same calculation in `main()` and a method.
10. Keep each method responsible for one clear task.

■ FINAL 15-MINUTE CHALLENGE

Students modify the project without receiving exact code:

1. Add **studyHours** input.
2. Create **calculateStudyScore()**.
3. Include it in the final score.
4. Add one overloaded **printReport()**.
5. Add one repeated category using a loop.
6. Explain one method to the class.

Exit Questions

What is a parameter vs an argument? Why use return? When use void? Why static here? What is local scope? Where did you use loop/break? Which method calls another method? Why overload? What happens if main() becomes huge?

■ MASTER FLOW: Input → Process → Method → Return → Reuse → Combine → Test